

Kubernetes Complete Guide

From Initial Setup to Full Production Deployment

A comprehensive, step-by-step reference covering all 14 phases of Kubernetes — Setup, Docker, Cluster, Namespace, Deploy, Service, Config, Scale, Monitor, Debug, Storage, RBAC, Ingress, and Rollout.

Table of Contents

Phase	Topic	Steps
1	Setup	Install kubectl, minikube, Helm
2	Docker	Dockerfile, build & push image
3	Cluster	Start cluster, view cluster info
4	Namespace	Create and manage namespaces
5	Deploy	Deployment YAML, apply, get pods, describe
6	Service	Service types, port-forward
7	Config	ConfigMap, Secrets
8	Scale	Manual scaling, HPA
9	Monitor	Logs, top, events
10	Debug	exec, troubleshoot
11	Storage	PersistentVolume, PVC
12	RBAC	ServiceAccount, Role, RoleBinding
13	Ingress	Ingress controller, TLS, routing
14	Rollout	Rolling updates, rollback, delete

Phase 1 — Setup

Step 1: Install kubectl

kubectl is the primary CLI to interact with any Kubernetes cluster.

```
bash

# macOS
brew install kubectl

# Linux
curl -LO "https://dl.k8s.io/release/${curl -sL https://dl.k8s.io/release/stable.txt}/bin/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/

# Windows (chocolatey)
choco install kubernetes-cli

# Verify installation
kubectl version --client
kubectl version
```

kubectl works with any Kubernetes cluster — local (minikube), cloud (EKS, AKS, GKE), or on-premise (OpenShift).

Step 2: Install minikube

minikube runs a single-node Kubernetes cluster on your laptop for development.

```
bash
```

```
# macOS
brew install minikube

# Linux
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Windows
choco install minikube

# Verify
minikube version
```

minikube is perfect for development and testing before deploying to cloud. It supports most Kubernetes features including Ingress, metrics-server, and the Dashboard.

Step 3: Install Helm

Helm is the standard package manager for Kubernetes — think of it like npm for Kubernetes.

```
bash
```

```
# macOS
brew install helm

# Linux
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Windows
choco install kubernetes-helm

# Verify
helm version

# Add common chart repositories
helm repo add stable https://charts.helm.sh/stable
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search for a chart
helm search repo bitnami/mysql

# Install a chart
helm install my-mysql bitnami/mysql --namespace bank-app

# List installed releases
helm list -n bank-app

# Uninstall a release
helm uninstall my-mysql -n bank-app
```

Phase 2 — Docker

Step 4: Dockerfile for Spring Boot

```
dockerfile
```

```
# Multi-stage build – builder compiles, runtime only contains JAR
FROM eclipse-temurin:17-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Multi-stage build keeps the final image small — builder stage compiles, runtime stage only contains the JAR and JRE. Final image is ~180MB vs ~500MB with full JDK.

Step 5: docker build & push

```
bash
```

```
# Build the Docker image
docker build -t myapp:1.0.0 .
docker build -t myapp:latest .

# Tag for Docker Hub
docker tag myapp:1.0.0 yourusername/myapp:1.0.0

# Tag for AWS ECR
docker tag myapp:1.0.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0.0

# Tag for Azure ACR
docker tag myapp:1.0.0 myregistry.azurecr.io/myapp:1.0.0

# Push to Docker Hub
docker push yourusername/myapp:1.0.0

# Push to AWS ECR (login first)
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
    123456789.dkr.ecr.us-east-1.amazonaws.com
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0.0

# Push to Azure ACR
az acr login --name myregistry
docker push myregistry.azurecr.io/myapp:1.0.0

# List local images
docker images

# Remove an image
docker rmi myapp:1.0.0
```

Always tag images with a version number, not just `latest`. Using `latest` in production makes rollbacks impossible because you can't track which version is running.

Phase 3 — Cluster

Step 6: minikube start / stop

```
bash
```

```
# Start minikube with specific resources
minikube start --cpus=4 --memory=8192 --disk-size=20g

# Start with specific Kubernetes version
minikube start --kubernetes-version=v1.29.0

# Start with Docker driver (recommended)
minikube start --driver=docker

# Check cluster status
minikube status

# Stop cluster (preserves state)
minikube stop

# Delete cluster (removes everything)
minikube delete

# Enable addons
minikube addons enable ingress
minikube addons enable metrics-server
minikube addons enable dashboard

# Open Kubernetes dashboard in browser
minikube dashboard

# Get minikube IP
minikube ip

# Use minikube's built-in docker (no push needed)
eval $(minikube docker-env)

# List all addons
minikube addons list
```

After `eval $(minikube docker-env)`, any `docker build` command builds directly into minikube's registry — no push/pull needed for local testing.

Step 7: kubectl cluster-info

```
bash
```

```
# Cluster info
kubectl cluster-info
kubectl cluster-info dump

# View all nodes
kubectl get nodes
kubectl get nodes -o wide          # with IPs and OS info

# Describe a specific node
kubectl describe node minikube

# Check component health
kubectl get componentstatuses

# View current context (which cluster you're connected to)
kubectl config current-context

# List all contexts (all clusters)
kubectl config get-contexts

# Switch cluster context
kubectl config use-context my-cluster
kubectl config use-context minikube
kubectl config use-context arn:aws:eks:us-east-1:123456:cluster/my-cluster

# View full kubeconfig
kubectl config view

# Set default namespace for current context
kubectl config set-context --current --namespace=bank-app
```

Always verify your current context before running commands. Accidentally running `kubectl delete` against production instead of staging is a real risk.

Phase 4 — Namespace

Step 8: Namespaces

```
bash
```



```
# List all namespaces
kubectl get namespaces
kubectl get ns

# Create a namespace (imperative)
kubectl create namespace bank-app
kubectl create namespace bank-staging
kubectl create namespace bank-prod

# Create namespace via YAML
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Namespace
metadata:
  name: bank-app
  labels:
    team: backend
    env: development
EOF

# Set default namespace for current context
kubectl config set-context --current --namespace=bank-app

# Delete namespace (deletes ALL resources inside!)
kubectl delete namespace bank-staging

# Get all resources in a namespace
kubectl get all -n bank-app

# Get resources across ALL namespaces
kubectl get pods --all-namespaces
kubectl get pods -A
kubectl get all -A
```

Always use namespaces to isolate environments (dev/staging/prod) and teams. Never deploy your app to the `default` namespace in production — use dedicated namespaces like `bank-prod`.

Phase 5 — Deploy

Step 9: Deployment YAML

yaml

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bank-app
  namespace: bank-app
  labels:
    app: bank-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: bank-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # max extra pods during update
      maxUnavailable: 0   # keep all current pods running
  template:
    metadata:
      labels:
        app: bank-app
    spec:
      containers:
      - name: bank-app
        image: yourusername/bank-app:1.0.0
        ports:
        - containerPort: 8080
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        env:
        - name: SPRING_PROFILES_ACTIVE
          value: "prod"
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: bank-secrets
              key: db-password
```

```
- name: APP_ENV
  valueFrom:
    configMapKeyRef:
      name: bank-config
      key: APP_ENV
livenessProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  initialDelaySeconds: 60 # Spring Boot needs time to start
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 5
  failureThreshold: 3
```

Always set resource requests and limits. Without limits a single pod can consume all node memory and crash everything. For Spring Boot add `initialDelaySeconds: 60` — the JVM needs time to start.

Step 10: kubectl apply / create

```
bash
```

```
# Apply a YAML file (creates or updates – idempotent)
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml

# Apply all YAML files in a directory
kubectl apply -f ./k8s/
kubectl apply -f ./k8s/ -R          # recursive

# Create (fails if already exists – use in scripts)
kubectl create -f deployment.yaml

# Apply from URL (GitOps pattern)
kubectl apply -f https://raw.githubusercontent.com/your/repo/main/k8s/deployment.yaml

# Dry run – preview without applying
kubectl apply -f deployment.yaml --dry-run=client
kubectl apply -f deployment.yaml --dry-run=server

# Validate YAML syntax
kubectl apply -f deployment.yaml --validate=true

# Apply with namespace override
kubectl apply -f deployment.yaml -n bank-prod

# Diff – show what will change before applying
kubectl diff -f deployment.yaml
```

Always use `apply` not `create` in CI/CD pipelines. `apply` is idempotent — safe to run multiple times. `create` fails if the resource already exists.

Step 11: kubectl get pods

```
bash
```

```
# List all pods in current namespace
kubectl get pods
kubectl get po                                # shorthand

# List with more info (node, IP)
kubectl get pods -o wide

# Watch pods in real time
kubectl get pods -w

# List pods with labels shown
kubectl get pods --show-labels

# Filter by label
kubectl get pods -l app=bank-app
kubectl get pods -l app=bank-app,tier=backend

# Get pods in specific namespace
kubectl get pods -n bank-app

# Get pods across all namespaces
kubectl get pods -A

# Output as JSON or YAML
kubectl get pod bank-app-xyz -o json
kubectl get pod bank-app-xyz -o yaml

# Get specific field with jsonpath
kubectl get pod bank-app-xyz -o jsonpath='{.status.podIP}'

# Count pods
kubectl get pods -l app=bank-app --no-headers | wc -l
```

Pod names have random suffixes (bank-app-7d4f9b-xkpz2). Always use label selectors (`-l app=bank-app`) to select groups of pods rather than individual pod names.

Step 12: kubectl describe

```
bash
```

```
# Describe a pod (shows events, errors, resource usage)
kubectl describe pod bank-app-7d4f9b-xkpz2
kubectl describe pod bank-app-7d4f9b-xkpz2 -n bank-app

# Describe a deployment
kubectl describe deployment bank-app
kubectl describe deployment bank-app -n bank-app

# Describe a node
kubectl describe node minikube

# Describe a service
kubectl describe service bank-app-svc

# Describe all pods matching label
kubectl describe pods -l app=bank-app

# Grep events section (most useful part)
kubectl describe pod bank-app-xyz | grep -A 20 Events
```

Common Events you will see:

Event	Meaning	Fix
ImagePullBackOff	Image not found or no credentials	Check image name/tag, add imagePullSecrets
OOMKilled	Container killed — memory limit exceeded	Increase memory limit
CrashLoopBackOff	Container crashing on startup	Check logs with <code>kubectl logs --previous</code>
Pending	Pod cannot be scheduled	Check node resources, taints
FailedMount	Volume cannot be mounted	Check PVC is Bound

The `Events` section at the bottom of `kubectl describe` is where 99% of deployment failures are explained. Always check this first when a pod won't start.

Phase 6 — Service

Step 13: Service YAML

yaml


```
# ClusterIP – internal only (default)
apiVersion: v1
kind: Service
metadata:
  name: bank-app-svc
  namespace: bank-app
spec:
  selector:
    app: bank-app          # matches pod labels
  ports:
    - port: 80             # service port
      targetPort: 8080     # container port
  type: ClusterIP

---
# NodePort – exposes on every node's IP (dev/test)
apiVersion: v1
kind: Service
metadata:
  name: bank-app-nodeport
spec:
  selector:
    app: bank-app
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080      # range: 30000-32767
  type: NodePort

---
# LoadBalancer – cloud provider creates external LB (production)
apiVersion: v1
kind: Service
metadata:
  name: bank-app-lb
spec:
  selector:
    app: bank-app
  ports:
    - port: 80
      targetPort: 8080
  type: LoadBalancer
```

Service types summary:

Type	Access	Use case
ClusterIP	Internal only	Service-to-service communication
NodePort	Node IP + port	Development, testing
LoadBalancer	External via cloud LB	Production (costs money per service)
Ingress	External via HTTP rules	Production (one LB for many services)

Step 14: Service commands

bash

```
# List all services
kubectl get services
kubectl get svc

# Get service details
kubectl describe service bank-app-svc

# Access NodePort service on minikube
minikube service bank-app-nodeport
minikube service bank-app-nodeport --url

# Port-forward for local testing (fastest way to test)
kubectl port-forward pod/bank-app-7d4f9b-xkpz2 8080:8080
kubectl port-forward service/bank-app-svc 8080:80
kubectl port-forward deployment/bank-app 8080:8080

# Access app via port-forward
curl http://localhost:8080/api/health
curl http://localhost:8080/actuator/health

# Expose a deployment quickly (imperative)
kubectl expose deployment bank-app \
  --port=80 --target-port=8080 --type=NodePort

# Check service endpoints (is service routing to pods?)
kubectl get endpoints bank-app-svc
kubectl describe endpoints bank-app-svc
```

`kubectl port-forward` is the fastest way to test your app locally without Ingress. If endpoints show `<none>`, the service selector doesn't match any pod labels.

Phase 7 — Config

Step 15: ConfigMap

```
bash
```

```
# Create ConfigMap from literal values
kubectl create configmap bank-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=INFO \
  --from-literal=DB_HOST=mysql-svc

# Create ConfigMap from properties file
kubectl create configmap app-properties \
  --from-file=application.properties
```

yaml

```
# ConfigMap YAML
apiVersion: v1
kind: ConfigMap
metadata:
  name: bank-config
  namespace: bank-app
data:
  APP_ENV: "production"
  LOG_LEVEL: "INFO"
  DB_HOST: "mysql-svc"
  application.properties: |
    server.port=8080
    spring.profiles.active=prod
    logging.level.com.bank=INFO
    spring.datasource.url=jdbc:mysql://mysql-svc:3306/bankdb
```

yaml

```
# Use in Deployment – as environment variables
env:
- name: APP_ENV
  valueFrom:
    configMapKeyRef:
      name: bank-config
      key: APP_ENV

# Use in Deployment – as mounted file
volumes:
- name: config-vol
  configMap:
    name: bank-config
volumeMounts:
- name: config-vol
  mountPath: /app/config

# Commands
kubectl get configmaps -n bank-app
kubectl describe configmap bank-config
kubectl edit configmap bank-config      # edit in place
kubectl delete configmap bank-config
```

ConfigMaps are for non-sensitive data only — database URLs, ports, feature flags, log levels.
Never put passwords, API keys, or tokens in ConfigMaps — use Secrets instead.

Step 16: Secrets

```
bash
```

```
# Create Secret from literals (auto base64 encoded)
kubectl create secret generic bank-secrets \
  --from-literal=db-password=MySuperSecretPass \
  --from-literal=api-key=sk-prod-abc123

# Create TLS secret
kubectl create secret tls bank-tls \
  --cert=./certs/tls.crt \
  --key=./certs/tls.key

# Create Docker registry secret (for private image pulls)
kubectl create secret docker-registry regcred \
  --docker-server=registry.azurecr.io \
  --docker-username=myuser \
  --docker-password=mypassword
```

yaml

```

# Secret YAML (values must be base64 encoded)
apiVersion: v1
kind: Secret
metadata:
  name: bank-secrets
  namespace: bank-app
type: Opaque
data:
  db-password: TX1TdXB1clNlY3JldFBhc3M=    # base64 encoded
  api-key: c2stcHJvZC1hYmMxMjM=

# Encode value: echo -n "MyPassword" | base64
# Decode value: echo -n "TX1QYXNzd29yZA==" | base64 -d

# Use in Deployment
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: bank-secrets
      key: db-password

# Use private registry for image pull
spec:
  imagePullSecrets:
  - name: regcred
  containers:
  - name: bank-app
    image: registry.azurecr.io/bank-app:1.0.0

# Commands
kubectl get secrets -n bank-app
kubectl describe secret bank-secrets    # shows keys, not values
kubectl delete secret bank-secrets

```

In production use HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault. Kubernetes Secrets are only base64 encoded (not encrypted) unless you enable encryption at rest.

Phase 8 — Scale

Step 17: Manual Scaling

```
bash

# Scale to specific number of replicas
kubectl scale deployment bank-app --replicas=5
kubectl scale deployment bank-app --replicas=5 -n bank-app

# Scale multiple deployments at once
kubectl scale deployment bank-app payment-svc --replicas=3

# Scale via patch
kubectl patch deployment bank-app -p '{"spec":{"replicas":5}}'

# Watch scaling in real time
kubectl get pods -w

# Check rollout status
kubectl rollout status deployment/bank-app

# Get current replica count
kubectl get deployment bank-app

# Scale to zero (stops all pods, keeps deployment)
kubectl scale deployment bank-app --replicas=0

# Scale back up
kubectl scale deployment bank-app --replicas=3
```

Scaling to 0 replicas stops all pods without deleting the Deployment. Use this during maintenance windows. The Deployment configuration and history are preserved.

Step 18: HorizontalPodAutoscaler (HPA)

```
bash

# Create HPA imperatively
kubectl autoscale deployment bank-app \
  --min=2 --max=10 --cpu-percent=70
```


yaml

```
# HPA YAML – scale on CPU and memory
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bank-app-hpa
  namespace: bank-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bank-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300 # wait 5min before scaling down
```

bash

```
# Apply HPA
kubectl apply -f hpa.yaml

# Check HPA status
kubectl get hpa
kubectl describe hpa bank-app-hpa

# Delete HPA
kubectl delete hpa bank-app-hpa

# Enable metrics-server (required for HPA)
minikube addons enable metrics-server
```

HPA requires metrics-server to be running. If HPA shows `unknown` for current metrics, metrics-server is not installed or not ready yet.

Phase 9 — Monitor

Step 19: kubectl logs

```
bash
```

```
# Get logs from a pod
kubectl logs bank-app-7d4f9b-xkpz2

# Stream logs in real time (-f = follow)
kubectl logs -f bank-app-7d4f9b-xkpz2

# Last N lines only
kubectl logs --tail=100 bank-app-7d4f9b-xkpz2

# Logs from previous crashed container
kubectl logs --previous bank-app-7d4f9b-xkpz2

# Logs from specific container (multi-container pod)
kubectl logs bank-app-7d4f9b-xkpz2 -c bank-app

# Logs from ALL pods matching label (production best practice)
kubectl logs -l app=bank-app --all-containers

# Logs with timestamps
kubectl logs -l app=bank-app --timestamps

# Last hour of logs
kubectl logs --since=1h bank-app-7d4f9b-xkpz2

# Save logs to file
kubectl logs bank-app-7d4f9b-xkpz2 > pod-logs.txt

# Grep logs for errors
kubectl logs -l app=bank-app | grep -i error
kubectl logs -l app=bank-app | grep -i exception
```

In production use ELK Stack (Elasticsearch + Logstash + Kibana) or Grafana Loki for log aggregation, searching, and long-term retention.

Step 20: kubectl top & events

```
bash
```

```
# View node resource usage
kubectl top nodes

# View pod resource usage
kubectl top pods
kubectl top pods -n bank-app
kubectl top pods --all-namespaces

# Sort by CPU usage
kubectl top pods --sort-by=cpu

# Sort by memory usage
kubectl top pods --sort-by=memory

# Watch resource usage continuously
watch kubectl top pods

# Cluster-wide events (sorted by time)
kubectl get events
kubectl get events -n bank-app
kubectl get events --sort-by='.lastTimestamp'
kubectl get events -w          # watch in real time

# Events for specific resource
kubectl get events --field-selector involvedObject.name=bank-app-xyz

# Warning events only
kubectl get events --field-selector type=Warning
```

`kubectl top` requires metrics-server. Install with: `minikube addons enable metrics-server`

Phase 10 — Debug

Step 21: kubectl exec

```
bash
```

```
# Open interactive shell in a pod
kubectl exec -it bank-app-7d4f9b-xkpz2 -- /bin/sh
kubectl exec -it bank-app-7d4f9b-xkpz2 -- /bin/bash

# Run a single command
kubectl exec bank-app-7d4f9b-xkpz2 -- env
kubectl exec bank-app-7d4f9b-xkpz2 -- cat /app/config/application.properties
kubectl exec bank-app-7d4f9b-xkpz2 -- curl http://localhost:8080/actuator/health

# Exec into specific container (multi-container pod)
kubectl exec -it bank-app-7d4f9b-xkpz2 -c bank-app -- /bin/sh

# Run ephemeral debug container
kubectl debug -it bank-app-7d4f9b-xkpz2 \
  --image=busybox --target=bank-app

# Copy files from/to container
kubectl cp bank-app-7d4f9b-xkpz2:/app/logs/app.log ./app.log
kubectl cp ./config.yml bank-app-7d4f9b-xkpz2:/app/config/config.yml

# Test service-to-service connectivity
kubectl exec -it bank-app-xyz -- curl http://mysql-svc:3306
kubectl exec -it bank-app-xyz -- nslookup mysql-svc
kubectl exec -it bank-app-xyz -- wget -qO- http://payment-svc/actuator/health
```

`kubectl exec` is the most powerful debugging tool. Use it to verify env vars are injected, test network connectivity between services, and inspect config files inside running containers.

Step 22: Troubleshooting Commands

```
bash
```

```
# Pod stuck in Pending – check events
kubectl describe pod bank-app-xyz | grep -A 10 Events

# Pod CrashLoopBackOff – check previous container logs
kubectl logs --previous bank-app-xyz

# OOMKilled – pod killed due to memory limit
kubectl describe pod bank-app-xyz | grep -i oom
# Fix: increase memory limit in Deployment YAML

# ImagePullBackOff – check image name and credentials
kubectl describe pod bank-app-xyz | grep -i image
# Fix: verify image name/tag, add imagePullSecrets

# Pod stuck in Terminating – force delete
kubectl delete pod bank-app-xyz --force --grace-period=0

# See all failing pods across all namespaces
kubectl get pods -A | grep -v Running | grep -v Completed

# Check if service is routing to pods (endpoints)
kubectl get endpoints bank-app-svc

# Test DNS resolution inside cluster
kubectl run test-dns --image=busybox --rm -it --restart=Never \
  -- nslookup bank-app-svc.bank-app.svc.cluster.local

# Test HTTP connectivity between services
kubectl run test-curl --image=curlimages/curl --rm -it --restart=Never \
  -- curl http://bank-app-svc.bank-app.svc.cluster.local/actuator/health

# View resource usage and limits for all pods
kubectl describe nodes | grep -A 5 "Allocated resources"
```

Kubernetes DNS pattern:

```
<service-name>.<namespace>.svc.cluster.local
bank-app-svc.bank-app.svc.cluster.local
mysql-svc.bank-app.svc.cluster.local
```

Phase 11 — Storage

Step 23: PersistentVolume & PVC

yaml

```
# PersistentVolumeClaim – request storage
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: bank-db-pvc
  namespace: bank-app
spec:
  accessModes:
    - ReadWriteOnce          # one node can read/write
  resources:
    requests:
      storage: 10Gi
    storageClassName: standard # 'gp2' on AWS, 'managed-premium' on Azure

---
# Use PVC in a Deployment
spec:
  containers:
    - name: mysql
      image: mysql:8.0
      env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: bank-secrets
              key: db-password
      volumeMounts:
        - name: mysql-data
          mountPath: /var/lib/mysql
  volumes:
    - name: mysql-data
      persistentVolumeClaim:
        claimName: bank-db-pvc
```

bash

```
# Commands

kubectl get pv                # list PersistentVolumes
kubectl get pvc               # list PersistentVolumeClaims
kubectl describe pvc bank-db-pvc  # check PVC is Bound
kubectl get storageclass      # list storage classes

# Access modes
# ReadWriteOnce (RWO)  - one node read/write
# ReadOnlyMany (ROX)  - many nodes read only
# ReadWriteMany (RWX) - many nodes read/write (NFS only)
```

PVC status must be **Bound** before pods can use it. If it stays **Pending**, no PersistentVolume matched the storage class or access mode.

Phase 12 — RBAC

Step 24: Role-Based Access Control

```
yaml
```



```
# 1. ServiceAccount – identity for your application pods
```

```
apiVersion: v1
```

```
kind: ServiceAccount
```

```
metadata:
```

```
  name: bank-app-sa
```

```
  namespace: bank-app
```

```
---
```

```
# 2. Role – permissions within a namespace
```

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role
```

```
metadata:
```

```
  name: bank-app-role
```

```
  namespace: bank-app
```

```
rules:
```

```
- apiGroups: [""]
```

```
  resources: ["pods", "services", "configmaps"]
```

```
  verbs: ["get", "list", "watch"]
```

```
- apiGroups: ["apps"]
```

```
  resources: ["deployments"]
```

```
  verbs: ["get", "list", "watch", "update"]
```

```
---
```

```
# 3. RoleBinding – assign Role to ServiceAccount
```

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: RoleBinding
```

```
metadata:
```

```
  name: bank-app-rolebinding
```

```
  namespace: bank-app
```

```
subjects:
```

```
- kind: ServiceAccount
```

```
  name: bank-app-sa
```

```
  namespace: bank-app
```

```
roleRef:
```

```
  kind: Role
```

```
  apiGroup: rbac.authorization.k8s.io
```

```
  name: bank-app-role
```

```
---
```

```
# Use ServiceAccount in Deployment
```

```
spec:
```

```
  serviceAccountName: bank-app-sa
```

```
  containers:
```

```
- name: bank-app
  image: yourusername/bank-app:1.0.0
```

```
bash

# Commands
kubectl get serviceaccounts -n bank-app
kubectl get roles -n bank-app
kubectl get rolebindings -n bank-app

# Test permissions for a ServiceAccount
kubectl auth can-i get pods \
  --as=system:serviceaccount:bank-app:bank-app-sa

kubectl auth can-i delete pods \
  --as=system:serviceaccount:bank-app:bank-app-sa

# View current user permissions
kubectl auth can-i --list
```

Always use a dedicated ServiceAccount per application with minimal permissions — principle of least privilege. Never use the `default` ServiceAccount in production.

Phase 13 — Ingress

Step 25: Ingress Controller & Rules

```
bash

# Install NGINX Ingress Controller (cloud)
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy

# Install on minikube
minikube addons enable ingress

# Verify controller is running
kubectl get pods -n ingress-nginx
```

```
yaml
```

```
# Ingress YAML – HTTP routing + TLS
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: bank-app-ingress
  namespace: bank-app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - api.mybank.com
      secretName: bank-tls          # TLS secret
  rules:
  - host: api.mybank.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: bank-app-svc
            port:
              number: 80
      - path: /payments
        pathType: Prefix
        backend:
          service:
            name: payment-svc
            port:
              number: 80
      - path: /notifications
        pathType: Prefix
        backend:
          service:
            name: notification-svc
            port:
              number: 80
```

bash

```
# Commands
kubectl get ingress -n bank-app
kubectl describe ingress bank-app-ingress
kubectl get ingress -n bank-app -o wide

# Get external IP of ingress
kubectl get ingress bank-app-ingress -o jsonpath='{.status.loadBalancer.ingress[0].ip}'

# Test ingress (add to /etc/hosts for local testing)
echo "${minikube ip} api.mybank.com" | sudo tee -a /etc/hosts
curl https://api.mybank.com/api/health
```

Ingress replaces multiple LoadBalancer services with a single entry point. One cloud LoadBalancer routes traffic to many services by URL path or hostname — much cheaper than one LB per service.

Phase 14 — Rollout

Step 26: Rolling Updates & Rollback

```
bash
```

```
# Update image to new version (triggers rolling update)
kubectl set image deployment/bank-app \
  bank-app=yourusername/bank-app:2.0.0
kubectl set image deployment/bank-app \
  bank-app=yourusername/bank-app:2.0.0 -n bank-app

# Watch rollout progress
kubectl rollout status deployment/bank-app

# Pause rollout (inspect new pods before continuing)
kubectl rollout pause deployment/bank-app

# Resume paused rollout
kubectl rollout resume deployment/bank-app

# View rollout history
kubectl rollout history deployment/bank-app
kubectl rollout history deployment/bank-app --revision=3

# ROLLBACK – most important recovery command!
kubectl rollout undo deployment/bank-app

# Rollback to specific revision
kubectl rollout undo deployment/bank-app --to-revision=2

# Restart all pods (force re-pull same tag)
kubectl rollout restart deployment/bank-app

# Annotate rollout with change reason
kubectl annotate deployment/bank-app \
  kubernetes.io/change-cause="Release v2.0.0 – added payment service"
```

`kubectl rollout undo` is the fastest recovery from a bad deployment. It switches back to the previous ReplicaSet — takes seconds. Always test rollbacks in staging first.

Step 27: Delete Resources

```
bash
```

```
# Delete a specific pod (Deployment auto-creates a replacement)
kubectl delete pod bank-app-7d4f9b-xkpz2

# Delete a deployment (removes all managed pods)
kubectl delete deployment bank-app
kubectl delete deployment bank-app -n bank-app

# Delete from YAML file
kubectl delete -f deployment.yaml
kubectl delete -f ./k8s/

# Delete service
kubectl delete service bank-app-svc

# Delete all resources in a namespace
kubectl delete all --all -n bank-app

# Delete namespace (removes EVERYTHING inside)
kubectl delete namespace bank-staging

# Force delete stuck pod
kubectl delete pod bank-app-xyz --force --grace-period=0

# Delete multiple resource types at once
kubectl delete deployment,service,ingress -l app=bank-app

# Delete by label
kubectl delete pods -l app=bank-app
```

Deleting a pod managed by a Deployment doesn't stop the app — the Deployment immediately creates a new pod. To stop the app, delete the Deployment or scale to 0 replicas.

Complete kubectl Quick Reference

Resource Shortnames

Resource	Shortname
pods	po

Resource	Shortname
services	svc
deployments	deploy
replicasets	rs
namespaces	ns
configmaps	cm
persistentvolumeclaims	pvc
persistentvolumes	pv
horizontalpodautoscalers	hpa
ingresses	ing
serviceaccounts	sa

Most Used Commands Summary

```

bash

kubectl get pods -A                # all pods all namespaces
kubectl get all -n bank-app        # all resources in namespace
kubectl describe pod <name>        # diagnose any issue
kubectl logs -f -l app=bank-app    # stream all pod logs
kubectl exec -it <pod> -- /bin/sh   # shell into container
kubectl apply -f ./k8s/            # deploy everything
kubectl rollout undo deployment/bank-app # emergency rollback
kubectl scale deployment bank-app --replicas=5 # manual scale
kubectl port-forward svc/bank-app-svc 8080:80 # local testing
kubectl top pods --sort-by=memory   # find memory hogs

```

Complete Spring Boot on Kubernetes Checklist

- ✓ Dockerfile with multi-stage build
- ✓ Image pushed to registry with version tag
- ✓ Namespace created (not default)
- ✓ ConfigMap for app properties
- ✓ Secret for passwords and API keys
- ✓ Deployment with replicas >= 2

7. ☒ Resource requests and limits set
8. ☒ Liveness and readiness probes configured
9. ☒ Service (ClusterIP) for internal routing
10. ☒ Ingress for external HTTP access
11. ☒ HPA configured for auto-scaling
12. ☒ PVC for database persistent storage
13. ☒ ServiceAccount with minimal RBAC
14. ☒ Rolling update strategy configured
15. ☒ Rollback tested in staging

Generated for Java Spring Boot Banking Application — Kubernetes 1.29+ / kubectl 1.29+